Replicating FTS-Diffusion: Degeneracy of the Released Generation Pipeline

Deli Lin Eric Zhao Lapo Linossi Tom Haidinger
ETH Zurich
{dellin, ershao, llinossi, thaidinger}@student.ethz.ch

Abstract

We revisit FTS-Diffusion (ICLR 2024), a financial time-series generator built from scale-invariant subsequence clustering (SISC), pattern-conditioned diffusion, and learned pattern evolution, and try to reproduce it from the released code. In our runs the released sampling pipeline is near-degenerate: synthetic S&P 500 paths are one repeated motif tiled into an almost perfectly linear price ramp (linear-fit $R^2 \approx 1.0$), independent runs are nearly identical, and synthetic return volatility is about $20\times$ too small. We trace this to three choices in the released sampling code: a deterministic argmax rollout of the pattern chain, a one-unit decoder hidden state, and a commented-out magnitude normalization. We are explicit about confounds we cannot rule out: no checkpoints are shipped, and our training budget is reduced. Consistent with degeneracy, a continuous-rollout Train-on-Augmentation-Test-on-Real (TATR) setup appears to improve S&P 500 forecasting by up to 85%, yet the same protocol never beats its baseline on GOOG or ZC=F. Augmentation carrying real structure would help on all three assets; a gain limited to the one trending asset points to a scale coincidence. We also do not reproduce the Appendix B.3 SISC toy targets. We present these as observations from one reproduction attempt rather than a verdict on the method.

1 Introduction

Deep generative models are attractive for finance because historical market data are scarce, noisy, nonstationary, and not experimentally reproducible. Financial returns also exhibit heavy tails, weak raw-return autocorrelation, volatility clustering, and recurring shapes at irregular time scales [Cont, 2001]. Sequence generators try to preserve temporal structure with adversarial, optimal-transport, and score-based objectives [Yoon et al., 2019, Xu et al., 2020, Tashiro et al., 2021]; financial generators additionally need to respect stylized facts and economically meaningful dependence [Wiese et al., 2020]. FTS-Diffusion [Huang et al., 2024] addresses this by discovering variable-length motifs and generating new trajectories motif by motif.

We reproduce FTS-Diffusion and extend its evaluation; we do not propose a new generator. We retrace the released pipeline and try several generation protocols, asking whether any rollout produces non-degenerate samples. We also test whether the SISC toy-data targets reproduce. The architecture is sensible, but in our hands the released sampling pipeline degenerates: the synthetic series collapses to a near-linear price ramp built from one repeated motif (Section 4). Given that collapse, the apparent S&P 500 gain probably reflects a scale coincidence that does not transfer across assets. Since the authors released no trained checkpoints, we retrained every module ourselves at a reduced budget, and cannot fully separate that choice from the degeneracy we observe.

Contributions. We make three contributions. (i) A code-level diagnosis of the released sampling pipeline (Section 4): three specific choices that drive the synthetic series to a near-degenerate linear

Preprint.

ramp, plus the underspecifications we had to resolve. (ii) A 30-seed S&P 500 TATR study, with the cross-asset observation that the continuous-rollout gain does not transfer to GOOG or corn futures (ZC=F), which we read as a scale coincidence. (iii) A set of mismatches between the paper's stated design and the released implementation (Section 4.2), together with an empirical account of why the pattern chain collapses to a single motif. We also document a gap in reproducing the Appendix B.3 SISC toy-data targets. Throughout, we separate claims about the released artifacts from what would need the authors' checkpoints to settle.

2 Background: FTS-Diffusion

FTS-Diffusion decomposes financial time-series generation into three modules [Huang et al., 2024]. Scale-invariant subsequence clustering (SISC) segments a univariate series into variable-length motifs using dynamic time warping (DTW) [Sakoe and Chiba, 1978] and DTW barycenter-style centroid updates [Petitjean et al., 2011]. Each segment is summarized by a pattern identity p_m, a duration scale α_m, and a magnitude scale β_m. A pattern generation module (PGM) then produces concrete segments from these motif conditions; it pairs a scaling autoencoder (Scaling-AE), which encodes and rescales segment shapes, with a pattern-conditioned diffusion model (PCDM) trained on a DDPM-style denoising objective [Ho et al., 2020].

Long-horizon behavior is controlled by the pattern-evolution module (PEM), which learns

$$(p_m, \alpha_m, \beta_m) \mapsto (p_{m+1}, \alpha_{m+1}, \beta_{m+1}), \tag{1}$$

with a classification loss for the next pattern and regression losses for the next duration and magnitude scales. Although useful, this separation makes downstream results sensitive to how the learned state chain is initialized, reset, and divided into synthetic training blocks.

3 Replication protocol

Assets and splits. Our main datasets are daily Yahoo Finance close-price histories for S&P 500, GOOG, and ZC=F. S&P 500 spans 1980–2020 and yields 10,088 trading days and 2,282 post-SISC segments with $K = 14$. GOOG spans 2004–2020 with 3,776 trading days and 733 post-SISC segments with $K = 11$. ZC=F spans 2000–2020 with 4,764 trading days and 629 post-SISC segments with $K = 11$. We follow a chronological train/test design: motif extraction, fitted generator components, standardizers, and downstream predictors are fit only on their allowed training windows, and final scores are computed on held-out real data.

Downstream benchmark. Our main downstream benchmark is TATR: Train on Augmentation, Test on Real. We divide the held-out real slice into an initialization portion and a final real test portion. A one-day-ahead LSTM forecaster is trained on the real initialization portion plus Y synthetic trading years, where one synthetic year has 252 days and Y is swept across a fixed grid. We score with mean absolute percentage error (MAPE) on the final real test portion. We use paper-like LSTM settings: one-day ahead, window size 64, hidden size 32, two layers, MAE training loss, Adam with learning rate 10^{-2}, and 100 epochs for the primary S&P 500 runs.

Synthetic rollout variants. We compare three ways to produce the synthetic augmentation path, each sharing the same data budget. We use the descriptive name with the code identifier in parentheses.

- **Independent fixed blocks** (`authors`): independent 252-day blocks, each generated from the same fixed initial state.
- **Continuous chunked** (`split`): one continuous trajectory from the pattern-evolution chain, cut into 252-day chunks for downstream training.
- **Independent blocks with random init** (`random_init`): independent 252-day blocks, each from a randomly drawn initial segment.

These three protocols differ only in how the Markov chain transitions across blocks. Because the pattern-evolution module is the only component that carries long-horizon state across motifs, block connectivity is what they probe.

4 Degeneracy of the released generation pipeline

We reproduce FTS-Diffusion end to end from the released code. Before downstream scores, we report what the generator actually produces, because it determines how the later numbers should be read. In our runs the released sampling path is near-degenerate: the synthetic price series is a single repeated motif tiled into an almost perfectly straight line. We identify three code choices behind this, with file references into our working copy (`fts-diffusion-ref/`), then show the degeneracy on the saved trajectories. We treat these as observations about the released artifacts under our setup rather than proofs about the method; Section 4.5 lists the confounds we cannot rule out.

4.1 Three choices in the sampling code

(1) Deterministic argmax rollout. FTS-Diffusion rolls out the pattern-evolution module (PEM) by taking `argmax` for the next pattern and the next length, and the raw regression mean for the next magnitude, with no sampling (`fts-diffusion-ref/models/sampling.py`, lines 30, 33, 34):

```
pred_pattern   = torch.argmax(probabilities_pattern, dim=1).unsqueeze(0)
pred_length    = torch.argmax(probabilities_length,  dim=1).unsqueeze(0) + l_min
pred_magnitude = pred[:, n_patterns+range_length:].float()
```

An autoregressive map $\text{state}_{m+1} = f(\text{state}_m)$ that is deterministic must, iterated from a fixed initial state, converge to a fixed point or a short cycle. Because the paper does not specify the rollout policy (greedy vs. sampling), this greedy reading of the release is defensible, but it removes the stochasticity the Markov-chain description suggests.

(2) One-unit decoder hidden state. Scaling-AE uses an LSTM decoder whose hidden dimension is fed from `pgm_params['sae_hidden_dim']`, set to 1 in the released configuration (`model_params.py`; the decoder LSTM is defined in `scaling_autoencoder.py`). With a single hidden unit the decoder has almost no capacity, and we observe it collapsing to a fixed output that ignores both the conditioning pattern and the diffusion latent.

(3) Commented-out magnitude normalization. A sampling step that would rescale the generated shape to unit range before multiplying by the magnitude β is commented out (`sampling.py`, lines 53–55), so the emitted segment has magnitude $\beta \times$ (whatever range the decoder produced) rather than β. In our runs that range is on the order of 0.07, far below the intended scale, which removes the magnitude information β was meant to carry.

4.2 Mismatches with the paper

Beyond the sampling choices above, the released training code departs from the paper's stated design in three places.

(1) Duration trained as classification. Although the paper models the next pattern as multi-class classification and the duration α and magnitude β as regression, the pattern-evolution head treats duration as classification too: it emits `n_patterns` + `range_length` + 1 logits and trains the length slice (one logit per duration in 10–21) with a cross-entropy loss against the integer target length $-$ 10, exactly like the pattern (`pattern_evolution_module.py`, lines 77, 98, 105, 111). Only the magnitude is regressed, and with an L_1 loss; the L_2 `MSELoss` line is commented out (lines 78–79). Appendix D.1 of the original paper allows that treating the length as classification is feasible, yet its main text describes regression.

(2) Magnitude β ignored at generation. `generate` receives only the pattern and length (`pattern_generation_module.py`, line 74), with an author comment that magnitude control was dropped from the denoising step (line 104). As in Section 4.1(3), β is reapplied only as a post-hoc multiply on top of a commented-out unit-range normalization, so it scales an arbitrary-range output rather than a unit shape.

(3) Deterministic rollout. Generation rolls out the chain with the `argmax` of Section 4.1(1), whereas the paper's Section 3.1 describes a stochastic Markov chain $Q(\cdot \mid \cdot)$. In fairness, the paper's

Algorithm 2 also writes the transition as a deterministic map, so the contradiction is between the stochastic description and the implementation.

We also probed why that deterministic rollout collapses to one motif (`scripts/probe_pem_transition.py`). On the trained S&P 500 PEM the next-pattern softmax is near-uniform: its maximum is 0.11 to 0.17 against a uniform $1/14 \approx 0.071$, peaked only thinly on the data mode $p{=}12$. So `argmax` returns $p{=}12$ for 13 of the 14 input patterns, and sweeping the length and magnitude inputs it never leaves the high-frequency SISC patterns $\{0, 1, 2, 3, 6, 8, 12\}$. The collapse is `argmax` acting on an undertrained, near-uniform head (the next-pattern loss carries weight 0.05), not a learned absorbing transition; multinomial sampling from the same head would not collapse.

4.3 Generated trajectories

Figure 1 contrasts a real S&P 500 path with one released-pipeline synthetic path. That synthetic series is visually a straight line; a least-squares linear fit gives $R^2 \approx 0.99999$. Three quantitative observations accompany the figure. First, independent runs of a configuration are nearly identical, consistent with a deterministic rollout. Second, the PEM state sequence collapses (Figure 1, bottom): on S&P 500 a single pattern is used in 99.6% of segments (a fixed point), while GOOG and ZC=F settle into period-2 cycles. Third, inspecting the trained PGM decoder pattern by pattern, the decoder output is near-constant across all patterns (cross-pattern standard deviation < 0.001): an up-trend pattern and a down-trend pattern yield essentially the same segment. Together these tile one tiny upward-creeping segment across the horizon, producing exactly the linear ramp in Figure 1. The same holds on the other assets: GOOG and ZC=F continuous rollouts are also near-linear (linear-fit $R^2 \approx 0.99999$), since their period-2 chains alternate between two near-identical segments.

4.4 Underspecifications

Several choices that matter for the result are not pinned down by the paper. Table 1 lists the ones we encountered, separating the three that are load-bearing for the degeneracy above from the compute-driven confounds and the remaining setup choices. We kept the released values wherever they existed, so that the run reflects the artifact as published rather than our preferences. We intend this as an audit aid rather than an accusation: underspecification is common, but here it coincides with the failure modes.

Table 1: Paper-vs-release underspecifications we resolved. "Released" marks a value shipped in the released configuration that we kept; "ours" marks a choice we had to make.

Design choice	In paper?	Value used	Note
Scaling-AE decoder hidden dim	no	1 (released)	load-bearing
PEM rollout policy	no	argmax (released)	load-bearing
Magnitude normalization at sampling	implied	commented out (released)	load-bearing
PGM / PEM training epochs	no	30 / 60 (ours; released 400 / 1000)	confound
PCDM diffusion steps	no	30 (ours; released 100)	confound
SISC k-means seed	no	42 (released)	shared seed
GOOG / ZC=F downstream split	S&P only	62.5% train (ours)	cross-asset parity
Initial segment for generation	no	first test segment / random (ours)	protocol switch

4.5 Confounds

We stress that the evidence above concerns the released artifacts as configured and trained in our hands rather than the method in principle. Two confounds prevent a stronger claim. First, no trained checkpoints are shipped with the release, so every reproduction retrains from the released scripts and we cannot compare against the authors' own generations. Second, our training budget is reduced relative to the released defaults (PGM 30 vs. 400 epochs, PEM 60 vs. 1000, PCDM 30 vs. 100 steps), a compute-driven choice that a reviewer can reasonably challenge. Three non-exclusive explanations remain that we have not fully separated: undertraining, an unlucky seed, and the one-unit decoder capacity. A full-budget retrain across seeds would falsify or confirm the undertraining hypothesis;

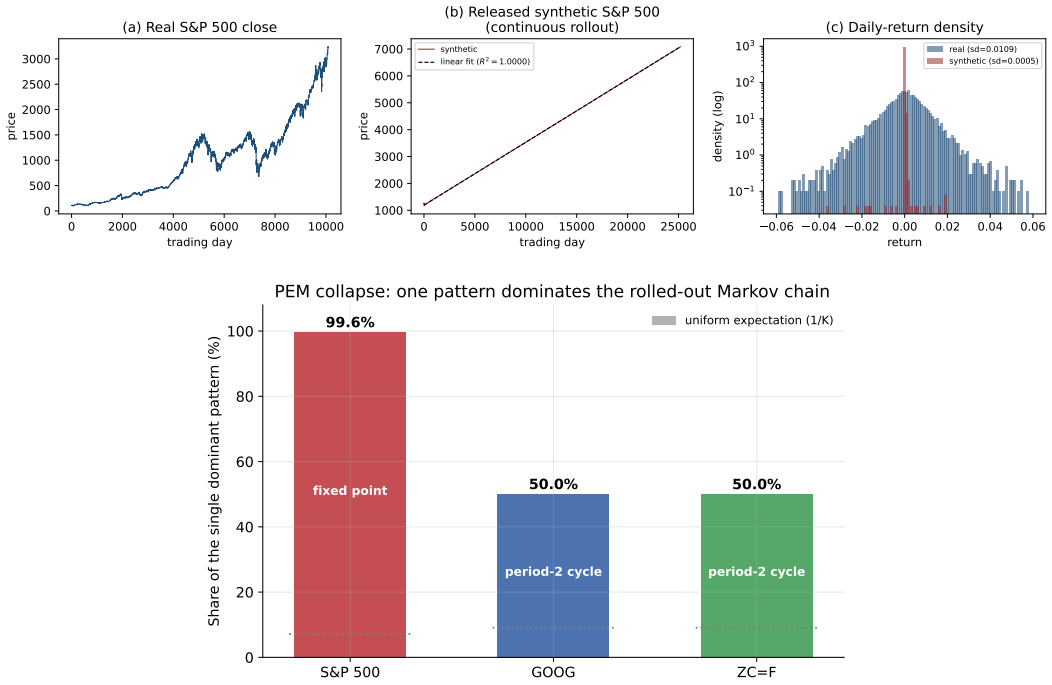


Figure 1: Released-pipeline degeneracy on S&P 500. Top, left to right: (a) real close prices show rich structure; (b) one synthetic continuous-rollout path is almost perfectly linear (linear-fit $R^2 \approx 0.99999$); (c) daily-return density on a log scale, where synthetic returns concentrate near a single value with standard deviation about $20\times$ smaller than real returns. Bottom: PEM state-sequence collapse under the deterministic rollout (from saved `run_*_states.npy`), the share of generated segments that use the single most frequent pattern. A single pattern covers 99.6% of segments on S&P 500 (K=14, a fixed point, 2520 segments) and 50% on GOOG (K=11, 2520 segments) and ZC=F (K=11, 1867 segments), both period-2 cycles; the dashed line marks the uniform expectation $1/K$ a non-degenerate chain would approach. We show one representative run; runs are nearly identical.

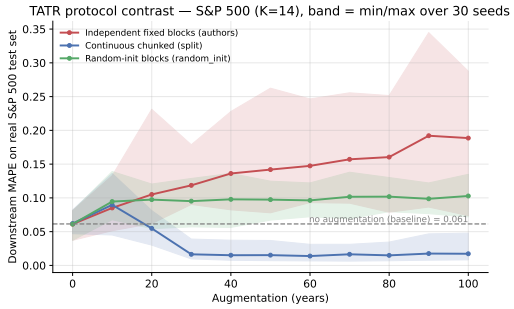
we flag it as the natural next experiment and do not assert a method-level conclusion without it. Given how unusual a one-unit decoder hidden state is for a published generative model, it would also be reasonable to ask the authors directly whether that value, and the commented-out magnitude normalization, correspond to the configuration behind the paper's headline results.

5 Replication results

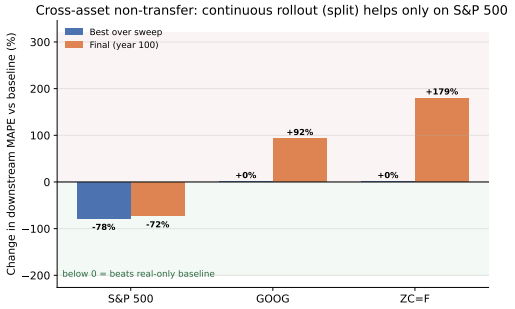
5.1 Protocol dependence of TATR

Huang et al. report that appending 100 synthetic years reduces one-day-ahead MAPE by 17.9%, 15.3%, and 17.4% on S&P 500, GOOG, and ZC=F, respectively. Our current S&P 500 evidence has three layers. First, the stored result matrices do not support that trend for the documented `authors` independent-block protocol: the stored S&P 500 `authors` curve finishes 208.4% worse than its baseline, while the continuous-trajectory curve finishes 71.6% better. Second, a one-seed protocol search found that continuous-rollout variants finish 76.5–85.4% better, whereas independent fixed blocks finish 96.5% worse. Third, Table 2 reports the primary 30-seed batch across the three protocols.

In that batch, the continuous chunked rollout reproduces a strong paper-like reduction in downstream MAPE, while both independent-block protocols worsen it. A single lucky seed does not drive this: across all 30 seeds the final continuous-chunked change stays below baseline (range $[-89.1\%, -21.2\%]$), whereas independent fixed blocks range $[+17.0\%, +438.3\%]$ and random-init



(a) Within-asset protocol contrast (S&P 500).



(b) Cross-asset non-transfer.

Figure 2: Downstream TATR depends on the synthetic-series protocol, and the S&P 500 gain does not transfer. (a) On S&P 500, continuous chunked (`split`) trajectories cross below the real-only baseline (dashed line marks the original 100-year TATR target; shaded bands show min/max across 30 seeds), while independent fixed blocks worsen as augmentation increases. (b) Change in downstream MAPE relative to each asset's no-augmentation baseline (below zero beats the real-only baseline): the `split` rollout helps only on S&P 500 (best -78%, final -72%); on GOOG and ZC=F it never beats baseline and finishes $+92\%$ and $+179\%$. A protocol carrying genuine structure would not flip sign across assets.

blocks $[+3.9\%, +263.3\%]$. Figure 2a shows the same contrast: continuous trajectories cross below the real-only baseline, while independent blocks worsen as augmentation increases.

Table 2: S&P 500 TATR summary over 30 seeds. Percentages are relative to each protocol's no-augmentation baseline; lower MAPE is better. "Best change" is the minimum over the augmentation sweep.

Protocol	Baseline	Best change	Final change
Independent fixed blocks (`authors`)	0.0611	$+0.0\%$	$+208.4\%$
Continuous chunked (`split`)	0.0620	-77.6%	-72.2%
Random-init blocks (`random_init`)	0.0611	$+0.0\%$	$+68.2\%$

Synthetic price levels explain the effect. Continuous trajectories start near 1253 and end near 7085 (mean level 4138); independent blocks start from the same level but repeatedly reset, ending near 1249 (mean 1214). Continuous rollouts thus drift the near-linear ramp through the scale of the upward-trending S&P 500 test period, so percentage error can fall simply because synthetic and real price levels overlap, not because the dynamics are informative. Independent blocks stay near the initial level, the overlap never happens, and error grows. Read with Section 4, this looks like a scale effect rather than useful augmentation. Cross-asset evidence below provides the cleaner test.

Train-on-Mixture-Test-on-Real (TMTR) controls, which train the forecaster on a fixed real-plus-synthetic mixture rather than sweeping the amount of augmentation, make the replication conclusion more conservative. In our TMTR runs, the reference curve reaches a best mean improvement of 26.8% but finishes 359.7% worse; a continuous offset-30 control reaches a best improvement of 50.0% and finishes 17.8% better; a continuous offset-50 control finishes 752.0% worse. So the evidence does not say that synthetic data always helps. A particular continuous TATR price-level rollout can create large apparent gains, while other augmentation and evaluation choices do not.

We make no claim of misconduct; the available generation details do not pin down the downstream result uniquely. Once the generator is seen to be degenerate, though, the simplest reading of the S&P 500 gain is a scale coincidence rather than augmentation that works under the right protocol. Either way, a reproducible benchmark should treat the Markov-chain rollout, initialization, block slicing, scaler fitting, and warm-up windows as first-class parameters, and should not rest a positive conclusion on downstream MAPE for a single trending asset.

GOOG and ZC=F test the coincidence reading most directly. We regenerated the stated real-asset SISC and architecture artifacts for both with $K = 11$, $l_{\min} = 10$, and $l_{\max} = 21$. A strict author-faithful downstream TATR rerun is blocked by the released S&P-style split logic, which consumes

$252 \times 5 = 1260$ held-out initialization points whereas the GOOG and ZC=F 80/20 held-out windows contain 744 and 963 points, so we adapted the split while keeping the proportions comparable. This changes the absolute MAPE level but not the sign of the augmentation effect, since every percentage is computed within the same split relative to that protocol's own no-augmentation baseline. Under that within-split comparison the effect is negative on both assets and for both protocol families (Table 3). Independent fixed blocks (`authors`) finish 1003% worse on GOOG and 75% worse on ZC=F. Nor does the continuous chunked (`split`) rollout that appears to help on S&P 500 transfer (Figure 2b): its best point over the whole sweep equals the no-augmentation baseline (it never beats it), and it finishes 92% worse on GOOG and 179% worse on ZC=F. If the generator carried useful structure, the protocol would not flip sign across assets; a gain confined to the one strongly trending asset points to a scale coincidence, not augmentation value. We therefore treat GOOG and ZC=F as covered for SISC/architecture setup and real-asset diagnostics rather than author-faithful downstream replications, but the within-split sign does not depend on author-faithfulness.

Table 3: Cross-asset TATR transfer. Final change in one-day-ahead MAPE relative to each asset's own no-augmentation baseline (lower is better); "best" is the minimum over the augmentation sweep. The continuous chunked (`split`) rollout beats the baseline only on S&P 500.

Asset	`authors` final	`split` best	`split` final
S&P 500	$+208.4\%$	-77.6%	-72.2%
GOOG	$+1003\%$	$+0.0\%$	$+92\%$
ZC=F	$+75\%$	$+0.0\%$	$+179\%$

5.2 SISC toy-data replication gap

We also stress-tested SISC on the Appendix B.3 simulated-data setting. Two metrics matter here. Per-unit DTW is the dynamic-time-warping distance between a recovered pattern centroid and its target shape, so lower is better. Jaccard is a segmentation-agreement measure, $|A \cap B|/|A \cup B|$, between the recovered set of segment boundaries A and the ground-truth set B: it equals 1.0 when the two segmentations coincide and 0 when they share no boundary, and we report a tolerance-2 variant that counts a recovered boundary as matched if it falls within two time steps of a true one. Huang et al. report one-pattern per-unit DTW/Jaccard of $0.009/0.938$ and multi-pattern average per-unit DTW/Jaccard of $0.01/0.784$. On our derived one-pattern runs, the best available DTW is 0.0202 and the best tolerance-2 boundary-Jaccard proxy is 0.8693. Across 18 successful multi-pattern runs varying seed, $l_{\max}$, and maximum iterations, our best average per-unit DTW was 0.0321, and our best author-style interval IoU (intersection over union of recovered and true segment intervals) was 0.6418. Table 4 reports both settings, and Figure 3 summarizes the multi-pattern gap.

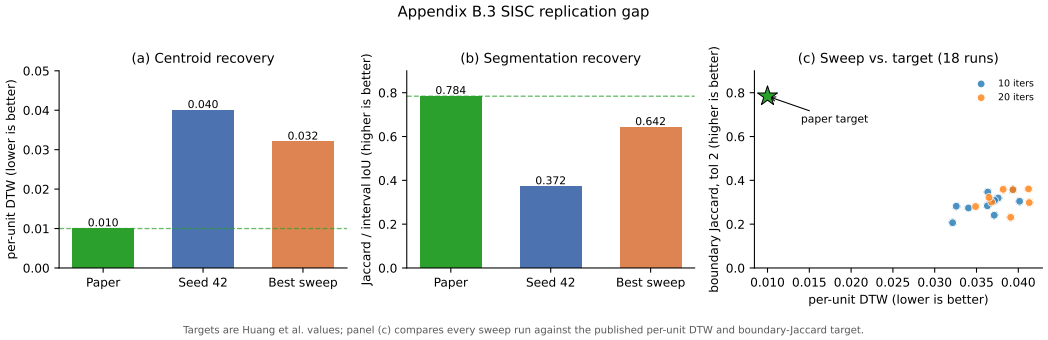


Figure 3: Appendix B.3 SISC replication gap on the toy data. (a) Centroid recovery (per-unit DTW, lower is better) and (b) segmentation recovery (Jaccard for the paper and our seed-42 run, interval IoU for the best sweep run; higher is better): our best multi-pattern runs do not reach the reported targets (dashed lines). (c) All 18 sweep runs (varying seed, $l_{\max}$, and maximum iterations) plotted as per-unit DTW against tolerance-2 boundary Jaccard; every run sits far from the published target (star), high in DTW and low in boundary agreement.

Table 4: Appendix B.3 SISC reproduction gap. Per-unit DTW is lower-is-better; the Jaccard/IoU column is higher-is-better. Reported values are from Huang et al.; ours are the best over our seed, $l_{\max}$, and iteration sweeps.

Setting	Source	Per-unit DTW	Jaccard / IoU
One-pattern	reported	0.009	0.938
One-pattern	ours	0.0202	0.869
Multi-pattern	reported	0.01	0.784
Multi-pattern	ours	0.0321	0.642

Increasing SISC iterations from 10 to 20 and changing $l_{\max}$ from 20 to 21 did not close the gap. Our best strict boundary Jaccard at tolerance 2 was 0.3608. Our best centroid metric remained about $3.2\times$ the reported target. We could not reproduce the exact toy benchmark, though this says nothing about the general idea of SISC: the released tree does not expose the exact one-pattern construction, synthetic generator, standard pattern arrays, seed state, or metric implementation used to produce Appendix B.3. More broadly, helper defaults matter: warm-up windows, augmentation scales, plotting conventions, and interval-overlap metrics can all change the apparent result if they go unaudited.

6 Limitations and broader impact

Our replication is constrained by the information and compute available for this study. Most importantly, the degeneracy analysis of Section 4 concerns the released artifacts as configured and as trained in our hands: no checkpoints are shipped, and our training budget is reduced relative to the released defaults. We have not run the full-budget, multi-seed retrain that would separate undertraining from an architectural cause, so we deliberately phrase those findings as observations about the release rather than conclusions about the method. Some runs use reference checkpoints rather than a full fresh retraining of all modules. GOOG and ZC=F have shorter histories than S&P 500, so the downstream split used for S&P 500 is not directly portable without changing the question being asked; we mitigate this by comparing within split, relative to each protocol's own baseline.

Synthetic financial data can be beneficial for stress testing, augmentation, and reproducibility, but it can also create misleading evidence if artificial predictability is mistaken for real market structure. Any release of synthetic paths should clearly label them as generated data, document the training period and generation protocol, and avoid implying that downstream forecasting gains translate into tradable performance.

7 Conclusion

FTS-Diffusion proposes a sophisticated architecture for financial time series: variable-length motifs, pattern-conditioned diffusion, and a learned motif chain. Working only from the code the authors released, we found that in several places the implementation does not match the paper. Its pattern-evolution module trains the duration as a classification although the paper specifies regression, and fits the magnitude with an L_1 loss rather than the L_2 the paper implies; the magnitude β never enters the diffusion conditioning and is reapplied over a commented-out range normalization, so it loses its meaning; and generation rolls out the chain with `argmax` rather than the stochastic Markov transition the paper describes. Its scaling-AE decoder also ships with a single hidden unit.

We believe these choices in the PGM and in the training procedure are logical faults that surface in the generated data: the synthetic series collapses to a near-linear ramp built from one repeated motif, and the only downstream gain we could reproduce is a scale coincidence on a single trending asset that does not transfer to GOOG or ZC=F. Without the authors' checkpoints we cannot fully separate the collapse from our reduced training budget, so we frame this as a study of the released artifacts. Those mismatches between the code and the paper are independent of the training budget.

References

Rama Cont. Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance*, 1(2):223–236, 2001.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020.

Hongbin Huang, Minghua Chen, and Xiao Qiao. Generative learning for financial time series with irregular and scale-invariant patterns. In *International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=CdjnzWsQax`.

Francois Petitjean, Alain Ketterlin, and Pierre Gancarski. A global averaging method for dynamic time warping, with applications to clustering. *Pattern Recognition*, 44(3):678–693, 2011.

Hiroaki Sakoe and Seibi Chiba. Dynamic programming algorithm optimization for spoken word recognition. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 26(1):43–49, 1978.

Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. CSDI: Conditional score-based diffusion models for probabilistic time series imputation. In *Advances in Neural Information Processing Systems*, volume 34, pages 24804–24816, 2021.

Magnus Wiese, Robert Knobloch, Ralf Korn, and Peter Kretschmer. Quant GANs: Deep generation of financial time series. *Quantitative Finance*, 20(9):1419–1440, 2020.

Tianlin Xu, Li K. Wenliang, Michael Munn, and Beatrice Acciaio. COT-GAN: Generating sequential data via causal optimal transport. In *Advances in Neural Information Processing Systems*, volume 33, pages 8798–8809, 2020.

Jinsung Yoon, Daniel Jarrett, and Mihaela van der Schaar. Time-series generative adversarial networks. In *Advances in Neural Information Processing Systems*, volume 32, 2019.